

Chapter 22: Elementary graph Algorithms

1) Searching a graph means systematically following the edges of a graph so as to visit the vertices of a graph.

2) Representations of graphs:

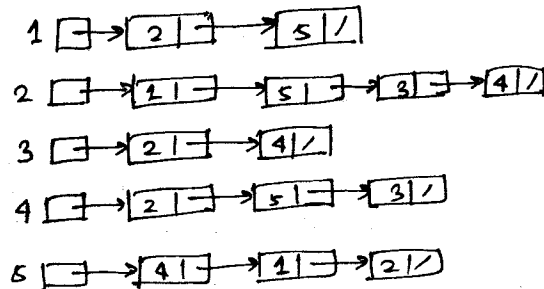
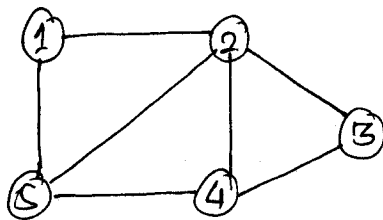
There are two standard ways to represent a graph (directed or undirected): -

- (i) as a collection of adjacency lists
- (ii) as an adjacency matrix

The adjacency list representation is usually preferred because it provides a compact way to sparse graphs - those graphs for which $|E|$ is much less than $|V|^2$.

An adjacency matrix representation may be preferred when the graph is dense i.e. $|E|$ is close to $|V|^2$, or when we need to be able to tell quickly if there is an edge connecting 2 vertices. ($|E|$ is always $<$ than $|V|^2$).

→ The adjacency list representation of a graph $G = (V, E)$ consists of an array Adj of $|V|$ lists. For each vertex $u \in V$ $Adj[u]$ consists of all the vertices adjacent to u in G . (It may contain pointers to these vertices). Vertices in an adjacency list are stored in an arbitrary order.



For any directed graph, sum of lengths of all adjacency lists = $|E|$. For any undirected graph, sum of lengths of all adjacency lists = $2|E|$.

For any graph, the amount of memory required by an adjacency list representation is $\Theta(V+E)$.

→ In a weighted graph, every edge has an associated weight, given by a weight function $w: E \rightarrow \mathbb{R}$. In an adjacency list representation the weight $w(u,v)$ of the edge $(u,v) \in E$ is simply stored with vertex v in u 's adjacency list.

→ A disadvantage of the adjacency list representation is that there is no quicker way to determine if a given edge (u,v) is present in the graph than to search for v in the adjacency list of u . This disadvantage can be remedied by an adjacency matrix representation of the graph, at the cost of using asymptotically more memory.

→ For an adjacency-matrix representation of a graph $G = (V, E)$, we assume that the vertices are numbered $1, 2, 3, \dots, |V|$ in some arbitrary manner. Then the adjacency matrix representation consists of a $|V| \times |V|$ matrix $A = (a_{ij})$ such that $a_{ij} = \begin{cases} 1, & \text{if } (i,j) \in E \\ 0, & \text{otherwise} \end{cases}$

The adjacency matrix representation requires $O(N^2)$ memory.

→ We can cut the memory required to store an adjacency matrix ^{representation of an undirected graph} by almost half.

The adjacency matrix of an undirected graph is its own transpose i.e. $A = A^T$ [$a_{ij} = a_{ji}$, since in an undirected graph (u,v) and (v,u) represent the same edge]. Thus, we only need to store the entries on and above the main diagonal.

→ We can use an adjacency matrix representation for weighted graphs as well. Rather than storing 0/1 as matrix entries we can store the weights of the edges. Non-existent edges may have NIL, ∞ or even 0 stored as the corresponding matrix entry.

→ If the graph is unweighted, the adjacency matrix representation offers an additional advantage.

Rather than using one word of computer memory per matrix entry, adjacency matrix can use just one bit per entry.

3) Breadth-first search:

Breadth first search is an algorithm for searching a graph that works on both directed and undirected graphs.

Given a graph $G = (V, E)$ and a distinguished source vertex s , breadth first search systematically explores the edges of G to "discover" every vertex that is reachable from s .

It also produces a "breadth-first tree" with root s and containing all the reachable vertices.

It locates a shortest path from the source vertex s to any reachable vertex v .

→ Breadth first search is so named because it expands the frontier between discovered and undiscovered vertices uniformly across the breadth of the frontier. The algorithm first discovers all vertices at distance K from s before discovering any vertices at distance $K+1$.

→ To keep track of progress, breadth-first search colors each vertex white, gray or black. Undiscovered vertices are painted white. Discovered vertices whose adjacency lists have not yet been fully examined are painted gray. Discovered vertices whose adjacency lists have been completely examined are painted black.

Gray vertices may have adjacent white vertices and represent the frontier between discovered and undiscovered vertices. Whenever a white vertex v is discovered in the course of scanning the adjacency list of an already discovered vertex u , the vertex v and the edge (u, v) are added to the ^{Breadth-first} tree. u is then said to be the predecessor of v in the breadth-first tree.

The procedure BFS assumes that the input graph is represented using adjacency lists. The color of every vertex u is stored in $color[u]$, the predecessor₁₀

Of u is stored in the variable $\pi[u]$. The distance from source s to vertex u is stored in $d[u]$.

It uses a queue Q to manage the set of gray vertices whose adjacency lists remain to be examined.

BFS(G, s)

```
1  for each vertex  $u \in V[G] - \{s\}$ 
2      do  $color[u] \leftarrow WHITE$ 
3          $d[u] \leftarrow \infty$ 
4          $\pi[u] \leftarrow NIL$ 
5   $color[s] \leftarrow GRAY$ 
6   $d[s] \leftarrow 0$ 
7   $\pi[s] \leftarrow NIL$ 
8   $Q \leftarrow \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11     do  $u \leftarrow DEQUEUE(Q)$ 
12        for each  $v \in Adj[u]$ 
13            do if  $color[v] = WHITE$ 
14                then  $color[v] \leftarrow GRAY$ 
15                    $d[v] \leftarrow d[u] + 1$ 
16                    $\pi[v] \leftarrow u$ 
17                   ENQUEUE( $Q, v$ )
18      $color[u] \leftarrow BLACK$ 
```

→ Depending upon the order in which the neighbours of a given vertex are visited by BFS, the breadth-first tree may vary but the distances computed by the algorithm will not.

→ Breadth first search runs in time that is linear in the size of the adjacency-list representation of G . After initialization, no vertex is ever whitened. This ensures that each vertex is enqueued at most once, and hence dequeued at most once.

Since enqueueing and dequeuing take $O(1)$ time, the total time devoted to queue operations is $O(V)$.

Because the adjacency list of each vertex is scanned just once, when the vertex is dequeued, each adjacency list is scanned at most once. Since the sum of the lengths of adjacency lists is $\Theta(E)$, total time spent in scanning adjacency lists is $O(E)$. The overhead for initialization is $O(V)$. Thus, the total running time of BFS is $O(V+E)$.

→ Let $\delta(s, v)$ be the shortest-path distance from s to v = minimum no. of edges in ~~the shortest~~ any path from vertex s to vertex v . $\delta(s, v) = \infty$ if there is no path from s to v .

lemma

Let $G = (V, E)$ be any directed or undirected graph, and $s \in V$ be an arbitrary vertex. Then, for any edge $(u, v) \in E$:- $\delta(s, v) \leq \delta(s, u) + 1$

Proof: ① If u is reachable from s , then so is v . In this case, the shortest path from s to v cannot be

longer than the shortest path from s to u followed by the edge (u, v) . [Otherwise you can always let the shortest path from s to v equal the shortest path from s to u + the edge (u, v)]. Thus the inequality holds.

(ii) If u is not reachable from s , $\delta(s, u) = \infty$ and certainly the inequality holds.

→ We want to show that BFS computes $d[u] = \delta(s, u)$ for every vertex $u \in V$. We first show that $d[u]$ bounds $\delta(s, u)$ from above.

Lemma

For every vertex $u \in V$, the value $d[u]$ computed by BFS satisfies $d[u] \geq \delta(s, u)$.

Proof: We use induction on the number of ENQUEUE operations. Our induction hypothesis is that $d[u] \geq \delta(s, u)$ for every $u \in V$.

The basis of induction is the situation immediately after s is enqueued. Inductive hypothesis holds here because $d[s] = 0 = \delta(s, s)$ and $d[u] = \infty \geq \delta(s, u)$ for all $u \in V - \{s\}$.

For the inductive step, consider a white vertex v that is discovered during a search from a vertex u . Inductive hypothesis implies that $d[u] \geq \delta(s, u)$.

We have:—

$$d[v] = d[u] + 1$$

$$\geq \delta(s, u) + 1$$

$$\geq \delta(s, v)$$

[From the 1st lemma]

Then vertex v is enqueued and is never enqueued again. Thus the value of $d[v]$ never changes again and the inductive hypothesis is maintained.

Lemma:

Suppose that during the execution of BFS on a graph $G=(V, E)$, the queue Q contains the vertices $\langle v_1, v_2, v_3, \dots, v_r \rangle$ where v_1 is the head and v_r is the tail. Then, $d[v_i] \leq d[v_{i+1}]$ for $i=1, 2, 3, \dots, r-1$

$$\text{and } d[v_r] \leq d[v_1] + 1.$$

→ Since, both $d[v_r] \geq d[v_1]$ and $d[v_r] \leq d[v_1] + 1$ hold true either $d[v_r] = d[v_1]$ or $d[v_r] = d[v_1] + 1$, implying that the queue can hold at most 2 distinct d values.

Corollary:

If a vertex v_i is enqueued before vertex v_j during the execution of BFS then $d[v_i] \leq d[v_j]$, implying that the d values in the queue are monotonically increasing over time.

Theorem: (Correctness of breadth first search)

Let $G=(V, E)$ be a directed or an undirected graph, and BFS is run on G from a given source vertex $s \in V$. Then, during its execution, BFS discovers every vertex $v \in V$ that is reachable, 19

from the source s , and upon termination,
 $d[u] = \delta(s, u)$ for all $u \in V$. Moreover, for any vertex
 $u \neq s$ that is reachable from s , one of the shortest
paths from s to u is a shortest path from s to $\pi[u]$
followed by the edge $(\pi[u], u)$.

Proof: Assume, for the purpose of contradiction,
that some vertex u receives a d value not equal
to its shortest path distance. [$u \neq s$, as $d[u] =$
 $\delta(s, s) = 0$]. From 2nd Lemma $d[u] \geq \delta(s, u)$. But
since $d[u] \neq \delta(s, u) \Rightarrow d[u] > \delta(s, u)$.

Vertex u must be reachable from s . Otherwise
 $\delta(s, u) = \infty \geq d[u]$.

Let w be the vertex that precedes u on a
shortest path from s to u . We must have $d[w] = \delta(s, w)$
since the contradiction is for u . Thus, we have:-

$$d[u] > \delta(s, u) = \delta(s, w) + 1 = d[w] + 1.$$

Consider the time when BFS chooses to dequeue
vertex w from the queue. At this time vertex u
may be either white, gray or black. We shall show that
in all these 3 cases we derive a contradiction to the
inequality.

If u is white, since u is adjacent to w , $d[u]$
would get the value $d[w] + 1$, contradicting the inequality.

If u is black, then it was already removed
from the queue and by the Corollary we have
 $d[u] \leq d[w]$, again contradicting the inequality.

If v is gray, then it was painted gray upon dequeuing some vertex w , which was removed from Q earlier than u and for which $d[u] = d[w] + 1$. By the corollary we have $d[w] \leq d[u]$ and hence $d[u] = d[w] + 1 \leq d[u] + 1$, once again contradicting the inequality.

Thus we conclude that $d[v] = \delta(s, v)$ for all $v \in V$. All vertices reachable from s must be discovered, for if they were not, they would have infinite d values. If $\pi[u] = u$, then $d[u] = d[u] + 1$. Thus, we can obtain a shortest path from s to v by taking a shortest path from s to $\pi[u]$ and then traversing the edge $(\pi[u], v)$.

→ Breadth-first trees:

The procedure BFS builds a breadth-first tree as it searches the graph. This tree is represented by the π field in each vertex.

Predecessor Subgraph: For any graph $G = (V, E)$, we define the predecessor subgraph of G as $G_\pi = (V_\pi, E_\pi)$ where

$$V_\pi = \{ v \in V : \pi[v] \neq \text{NIL} \} \cup \{s\}$$

and

$$E_\pi = \{ (\pi[v], v) : v \in V_\pi - \{s\} \}$$

Lemma: When the procedure BFS is applied to any directed or undirected graph $G = (V, E)$, BFS constructs π so that the predecessor subgraph 16

$G_\pi = (V_\pi, E_\pi)$ is a breadth-first tree.

→ The following procedure prints out the vertices on a shortest path from s to u :-

PRINT-PATH(G, s, u)

```
1  if  $v = s$ 
2      then print  $s$ 
3  else if  $\pi[u] = \text{NIL}$ 
4      then print "no path from"  $s$  "to"  $u$  "exists"
5      else PRINT-PATH( $G, s, \pi[u]$ )
6      print  $u$ 
```

4) Depth-first Search:

Depth-first search searches deeper in the graph whenever possible. The strategy is to discover any vertex reachable from the most recently discovered vertex, before discovering any other vertex reachable from the parent of the most recently discovered vertex.

This process continues until we have discovered every vertex reachable from the original source vertex. If any undiscovered vertices remain in the graph, then one of them is selected as the new source vertex and the search is repeated. This entire process is repeated until all vertices are discovered.

→ Conceptually, breadth-first search could proceed from multiple sources and depth-first search could be limited to one source. But our approach reflects how results of these searches are typically used. Breadth-first search is usually used to find shortest-path distances from a given source. Depth-first search is often a subroutine in another algorithm.

→ The predecessor subgraph produced by breadth-first search produces a single breadth-first tree. But the predecessor subgraph produced by depth-first search forms a depth-first forest composed of several depth-first trees.

The predecessor subgraph of a depth-first search is defined as $G_\pi = \{V, E_\pi\}$ where

$$E_\pi = \{ (\pi[v], v) : v \in V \text{ and } \pi[v] \neq \text{NIL} \}$$

→ Depth-first search colors every undiscovered vertex white. A vertex is colored gray when it is discovered (but its adjacency list has not been fully traversed).

A vertex is blackened when it is finished, that is its adjacency list has been fully traversed. This technique guarantees that every vertex ends up in a single depth-first tree, so that these trees are disjoint.

→ Depth-first search maintains two timestamps with every vertex:

① $d[v]$ → records the time of first discovery of the vertex

(i) $f[v]$ records the time when DFS finishes examining v 's adjacency list.

Since there are just 2 events associated with a vertex: (i) discovery of the vertex, and (ii) completion of examination of the vertex's adjacency list, and since there are $|V|$ vertices, timestamps are integers between 1 and $2|V|$.

Any vertex is white before time $d[v]$, gray between time $d[v]$ and $f[v]$ and BLACK thereafter.

DFS(G) $\triangleright$ G may be directed or undirected

```
1 for each vertex  $u \in V[G]$ 
2   do  $color[u] \leftarrow WHITE$ 
3    $\pi[u] \leftarrow NIL$ 
4  $time \leftarrow 0$   $\triangleright$  global variable used for timestamping
5 for each vertex  $u \in V[G]$ 
6   do if  $color[u] = WHITE$ 
7     then DFS-VISIT( $u$ )
```

DFS-VISIT(u)

```
1  $color[u] \leftarrow GRAY$ 
2  $time \leftarrow time + 1$ 
3  $d[u] \leftarrow time$ 
4 for each  $v \in Adj[u]$ 
5   do if  $color[v] = WHITE$ 
6     then  $\pi[v] \leftarrow u$ 
7     DFS-VISIT( $v$ )
```

8 $color[u] \leftarrow BLACK$

9 $time \leftarrow time + 1$

10 $f[u] \leftarrow time$

→ The results of depth-first search may depend upon the order in which the source vertices are selected by DFS and upon the order in which the neighbours of a vertex are visited by DFS-VISIT. These different visitation orders tend not to cause problems in practice, as any depth-first search result can usually be used effectively, with essentially equivalent results.

→ The runtime of DFS is $\Theta(V+E)$, as DFS-VISIT is invoked exactly once for each vertex and traverses the adjacency list of every vertex exactly once. $\therefore$

5) Depth-first search is usually used to learn about the structure of a graph.

Parenthesis structure: The discovery and finishing times of the vertices of a graph exhibit parenthesis structure.

If you represent the discovery of a vertex u with a left parenthesis " $(u$ " and represent its finishing by a right parenthesis $)u$ ", then the history of discoveries and finishings makes a well-formed expression in the sense that the parentheses are properly nested.

Parenthesis theorem:

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any 2 vertices u, v

and v , exactly one of the following three conditions holds:

- i) the intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ are disjoint and neither u nor v is a descendant of the other in a depth-first forest,
- ii) the interval $[d[u], f[u]]$ ~~and~~ ^{is contained entirely within} $[d[v], f[v]]$, and u is a descendant of v in a depth-first tree, or
- iii) the interval $[d[v], f[v]]$ is contained entirely within the interval $[d[u], f[u]]$ and v is a descendant of u in a depth-first tree.

Proof: We begin with the case when $d[u] < d[v]$.

There are 2 subcases to consider:

- i) When $d[v] < f[u] \rightarrow$ means that v was discovered while u was still gray. This implies that v is a descendant of u . (Since, whenever DFS-VISIT is called on a vertex u , first all descendants of u are discovered, then the search proceeds to discover the siblings of u and their descendants).

Moreover, since v is a descendant of u , first the recursion of depth-first search should finish on v and then it finishes on u .

Hence, in this case, the interval $[d[v], f[v]]$ is contained entirely within the interval $[d[u], f[u]]$

- ii) When $d[v] > f[u]$ the intervals are disjoint. Neither vertex was discovered while the other vertex was gray and so neither vertex was a descendant

of the other.

The case when $d[v] < d[u]$ is similar.

White path theorem:

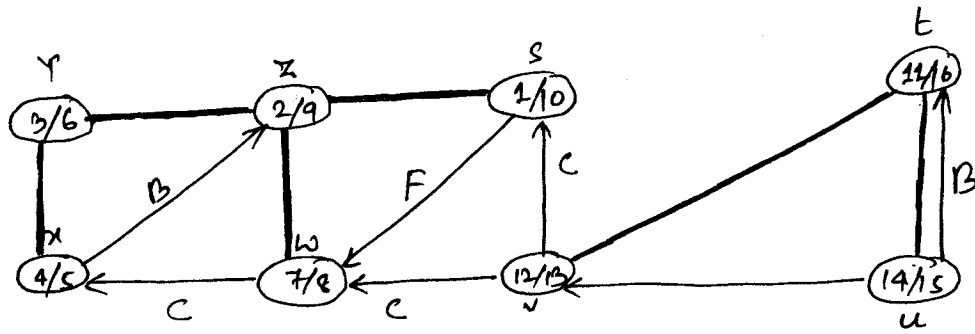
In a depth-first forest of a directed or undirected graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $d[u]$ that the search discovers u , vertex v can be reached from u along a path consisting entirely of white vertices.

6) Classification of edges in a depth-first search:

We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

- i) Tree edges are edges in the depth-first forest G_π .
Edge (u, v) is a tree edge if vertex v was discovered by exploring edge (u, v) .
- ii) Back edges are those edges (u, v) that connect a vertex u to an ancestor v in a depth-first tree.
Self-loops (which may occur in directed graphs) are considered to be back edges.
- iii) Forward edges are those non-tree edges (u, v) that connect a vertex u to a descendant v in a depth-first tree.
- iv) Cross edges are all other edges. They can go between vertices in the same depth-first tree,

as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.



The DFS algorithm can be modified to classify edges as it encounters them. When you explore the edge (u,v) , u must be gray and depending upon the color of v the edge may be classified as:-

- ① ~~WHITE~~ Tree edge if v is WHITE,
- ② Back edge if v is gray, and
- ③ Forward or Cross edge if v is Black.

Note that the gray vertices always form a linear chain of descendants corresponding to the stack of active DFS-VISIT invocations.

In the third case, it can be shown that (u,v) is a forward edge if $d[u] < d[v]$ and cross edge if $d[u] > d[v]$.

→ There may be some ambiguity in the type classification of an edge in an undirected graph, since (u,v) and (v,u) are really the same edge. In ^{such a} ~~that~~ case, the edge is classified as the first type in the classification list that applies. In other words, the edge is classified according to whichever of (u,v) and (v,u) is encountered first during the execution of the algorithm.

→ Theorem:

In a depth-first search of an undirected graph G , every edge is a tree edge or a back edge.

Proof: Let (u,v) be an arbitrary edge of G , and suppose without loss of generality that $d[u] < d[v]$. Then, v must be discovered ^{and finished} before we finish u .

If the edge (u,v) is explored first in the direction from u to v , then v is undiscovered (white) at that time, for otherwise we would have explored this edge already in the direction from v to u . Thus, (u,v) becomes a tree-edge.

If (u,v) is ^{explored} ~~discovered~~ first in the direction from v to u , then (u,v) is a back edge, since u is still gray at the time the edge is first explored.

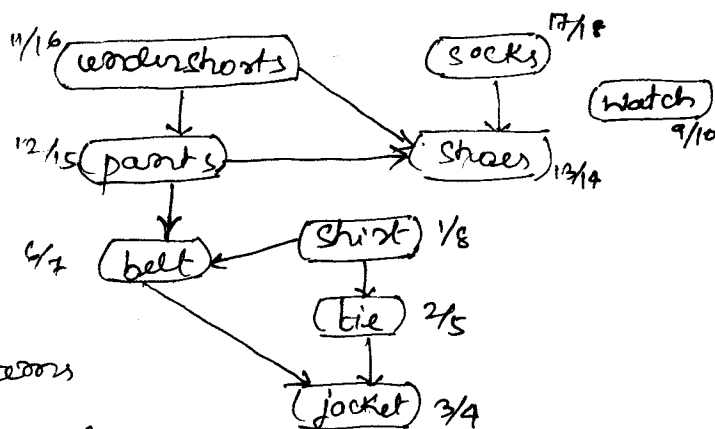
7) Applications of depth-first search:

1) Topological sort of a dag:

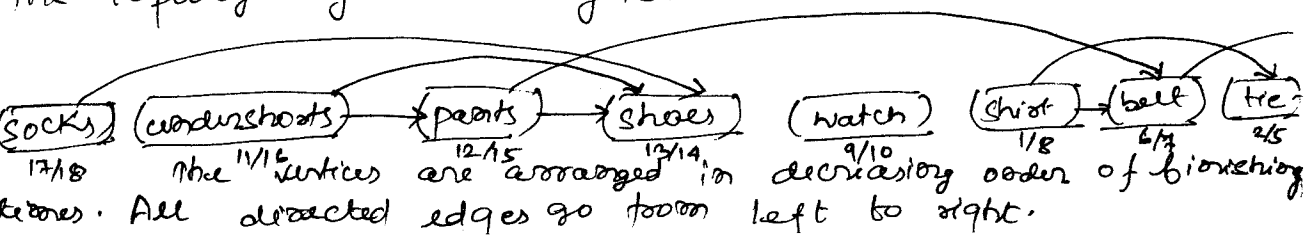
A topological sort of a dag $G=(V,E)$ is a linear ordering of all its vertices such that if G contains an edge (u,v) then u appears before v in the ordering. Topological sort can be viewed as an ordering of the vertices of the dag along a horizontal line so that all directed edges go from left to right.

→ Directed acyclic graphs are used by applications to indicate precedences among events.

For example, consider how an executive gets dressed in the morning. He must don certain garments before others (eg., socks before shoes). Other items may be put on in any order.



(eg., socks and pants). A directed edge (u,v) in the dag indicates that garment u must be donned before garment v . Thus, a topological sort of this dag gives an order for getting dressed. The topologically sorted dag is: -



TOPOLOGICAL-SORT (G)

- 1 call DFS(G) to compute finishing times $f[u]$ for each vertex u
- 2 as each vertex finishes insert it onto the front of a linked list
- 3 return the linked list of vertices

→ Topological sort can be performed on a dag in $\Theta(V+E)$ time [Since DFS takes $\Theta(V+E)$ time and it takes $O(1)$ time to insert each of the $|V|$ vertices onto the front of the linked list.]

Lemma

A directed graph is acyclic if and only if a depth-first search of it yields no back edges.

Proof: $\Rightarrow$: Suppose that there is a back edge (u,v) . The definition of back edge implies that v must be an ancestor of u in the depth-first forest. Since v is an ancestor of u , there must be a path from v to u . This coupled with the back-edge (u,v) completes a cycle.

$\Leftarrow$: Suppose that G contains a cycle C . We show that a depth-first search of G yields a back-edge. Let v be the first vertex to be discovered in C . Since C is a cycle we can assume an edge (u,v) without any loss of generality. At the time $d[v]$, the vertices of C form a path of white vertices from v to u .

By the white-path theorem vertex u becomes a descendant of v in the depth-first forest. Therefore, (u,v) is a back-edge.

2) Decomposition of a directed graph into its strongly connected components:

→ A strongly connected component of a directed graph is that maximal set of vertices, such that every pair of vertices in that maximal set is mutually reachable.

→ The transpose of a graph G is defined to be that graph which consists of the edges in G , with their directions reversed. If $G = (V, E)$ then $G^T = (V, E^T)$ where $E^T = \{(u, v) : (v, u) \in E\}$.

G and G^T will have exactly the same strongly connected components: if u and v are mutually reachable in G , they are mutually reachable in G^T as well.

STRONGLY-CONNECTED COMPONENTS (SCC)

- 1 Call DFS(G) to compute finishing times $f[u]$ for each vertex u
- 2 compute G^T
- 3 Call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing finishing times $f[u]$
- 4 Every depth-first tree in the depth-first forest acts as a separate connected component, so O/P its vertices as a separate connected component.

→ each DFS is $\Theta(V+E)$ and computation of G^T from G is also $\Theta(V+E)$. Hence, SCC is $\Theta(V+E)$.

Lemma

Let C and C' be distinct strongly connected components in a digraph G . If vertices $u, v \in C$ and vertices $u', v' \in C'$ and there is a path $u \rightsquigarrow u'$ in G , there cannot be a path from $v' \rightsquigarrow v$ in G .

Proof: If there is a path $v' \rightsquigarrow v$ in G , then there are paths $u \rightsquigarrow u' \rightsquigarrow v'$ and $v' \rightsquigarrow v \rightsquigarrow u$ in G .

Thus u and v' are mutually reachable, contradicting the assumption that C and C' are distinct strongly connected components.

→ In SCC we would always let $d[u]$ and $f[u]$ represent the discovery and finishing times of the 1st DFS.

If U is any set of vertices, we define $d(U)$ to be the earliest discovery time and $f(U)$ to be the latest finishing time of any vertex in U .

$$d(U) = \min_{u \in U} \{d[u]\} \quad \text{and} \quad f(U) = \max_{u \in U} \{f[u]\}$$

Lemma

If C and C' are distinct strongly connected components in a digraph G and there is an edge (u, v) in G from u in C to v in C' , then $f(C) > f(C')$.

Corollary

If C and C' are distinct strongly connected components in a digraph G and there is an edge (u, v) in G^T from u in C to v in C' then $f(C) < f(C')$.

The above corollary tells us that each edge in G^T that goes between different strongly connected components goes from a component with an earlier finishing time to a component with a later finishing time (in the 1st depth-first search).

→ The SCC procedure works fine because; when we start with the SCC C whose finishing time $f(C)$ is maximum, we discover every vertex that is reachable from the root of C , but there are no edges in G^T from C to any other SCC according to above corollary. When we next repeat the search on some other SCC C' , the only SCC that has an edge from C' is C , which we have already visited. Hence, each depth-first tree produced by the 2nd DFS is exactly one SCC.

→ **Component graph G^{SCC}** : By contracting all edges of a graph G into a single vertex whose incident vertices are within the same strongly connected component of G , we get G^{SCC} .

$$G^{SCC} = (V^{SCC}, E^{SCC})$$

V^{SCC} has one vertex v_i for every strongly connected component C_i of G .

E^{SCC} has one edge (v_i, v_j) if G contains a directed edge from some $x \in C_i$ to some $y \in C_j$.

→ The component graph of any graph is a dag.